

Introduction to using Transformer networks for macroeconomic analysis

Oliver Snellman
oliver.snellman@gmail.com

November 2025

Abstract

The transformer network is a machine learning method proposed in 2017 by Google Research ([Vaswani et al. 2017](#)). Due to its simplicity and power, the Transformer quickly became the new paradigm in computer science for modeling sequential data, such as natural language. Economic time series data is also inherently sequential, with some peculiar features. This document is an overview of the Transformer methodology for analyzing macroeconomic data, developed in my [dissertation](#) ([Snellman 2025](#)). The aim is to provide high-level intuitions with lots of visualizations, and not to delve into technicalities more than necessary. Sections [1](#) and [2](#) introduce economic modeling and model selection, Section [3](#) outlines three key challenges to using Transformers in macro modeling, and the rest of the sections address those challenges. More specifically, Section [4](#) outlines the Transformer algorithm for analyzing data instead of mere prediction, Section [5](#) shows how existing economic models can be used to guide the Transformer and improve performance with small datasets, and Section [6](#) demonstrates how the results from a Transformer can be analyzed with Attention-based custom tools. The document can also be listened to as a [Lectio-presentation](#) on YouTube.

1 Motivation

Economics is a social science studying human interaction and its emergent results. *Macroeconomics* focuses on large-scale phenomena, such as i) fluctuations in the economic conditions, or the business cycle, and ii) evaluating the accumulation of fragility in the financial system, which predisposes the whole economy to malfunction. It is typical to use statistical algorithms and time series data to study the macroeconomy. Time series data requires special care, as part of the information is related to the order of observations.

We have to constantly make consequential decisions regarding fiscal, monetary and macroprudential policy. These decisions impact the well-being of millions of people. Having

a better understanding of our economic predicament, where we are and where we are heading, *could* help us make better choices. Having more information does not dictate how one should act, but it can help in pursuing whatever value-laden goals one has. Information is useful! But obtaining useful information is hard. Situation analysis is challenging, because often what we are most interested in cannot be directly measured. For example, the business cycle or the fragility of the financial system cannot be directly observed. Information about these phenomena has to be inferred from macroeconomic data, by using a model. These are the two applications in the [dissertation](#) for the new Transformer methodology.

2 Model selection

We cannot access the information in data without a model. A model organizes, prioritizes and interprets observations. A model also helps to organize thoughts, check that they are coherent, and to communicate them to others. Here we concentrate on mathematical models, but also verbally expressed theories are models in the broad sense of the word. There are endless ways to specify the functional form for a model – the assumed structure of the phenomenon in question. A suitable model has to be selected separately for each task. This is the *model selection* problem.

As a thought experiment, consider the set of all possible models, or the model space, in Figure 1. Consider also a phenomenon that relates to several variables with certain connections between them. Let's call this the 'true model'.

- **First best:** Specify a model with the same structure as the true model, to study data related to the phenomenon.
 - Not possible in the real world, because true models are infinitely complex and unobservable.
- **Second best:** Select a useful model which adequately reflects the core dynamics of the phenomenon.

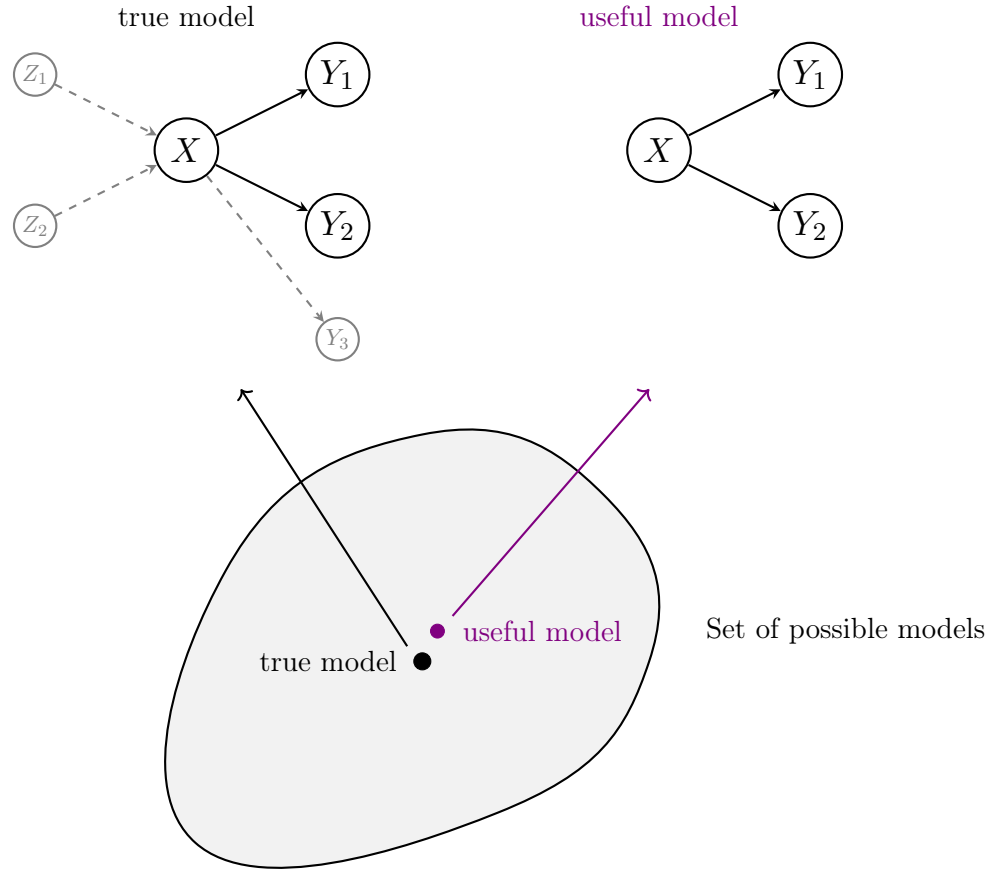


Figure 1: All models are wrong, but some of them are useful. Points in the grey set represent different models, and the closer they are to the 'true model' for the problem at hand, the more useful they are in studying that problem.

There are, broadly speaking, two approaches to model selection, visualized in Figure 2.

- **Conventional approach:** Researcher chooses the model based on convenience, expert knowledge and on preliminary exploration of data. This amounts to picking a point in the set of possible models and then adjusting it if needed. Often simplicity is favored, especially the linear model. What is crucial here is that the model has to be selected before it can be estimated with data.
- **Machine learning approach:** The model selection is outsourced to an algorithm, which can autonomously learn complex and useful ways to represent the data. This amounts to starting from the set of all possible models and then converging to a point. What is important here is that the model is selected and estimated at the same time by the algorithm, during training. However, the model is often only implied by the procedural algorithm, and it cannot be easily expressed in a closed form.

Most of the macroeconomic models rely on the conventional approach, even though the machine learning approach has the potential to improve model selection.

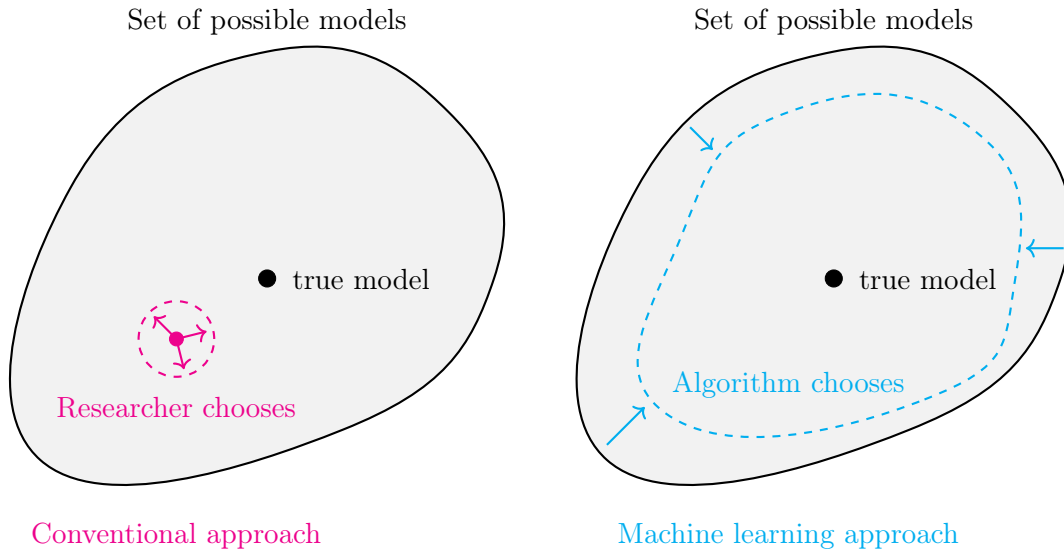


Figure 2: Two approaches to selecting a useful model from the set of alternatives.

Some *caveats*: Any point in the set of possible models can, in principle, be reached using both approaches. In other words, both the machine learning and the conventional toolkits include universal function approximators. Even though machine learning typically requires fewer a priori assumptions for selecting a model than the conventional protocol, machine learning can be more laborious when it comes to choosing hyperparameters and operating the training process. In practice, the set of models available is always restricted by computational constraints. The word 'true' is a tricky epistemological concept, and in some sense the only True model is the reality itself. It is nonetheless non-controversial to think that different models can be more or less useful for a task. Therefore, we could imagine a maximally useful model and label that as the 'true' one in a pragmatist sense.

3 Main challenges to machine learning

There are three key challenges, in my opinion, which explain why Transformers in particular and machine learning in general have not yet gained a larger footing in macroeconomic research.

1. Frontier machine learning methods, such as the Transformer neural network, are often perceived to be only useful for time series prediction tasks. Prediction is not the main interest of macroeconomics. This challenge is addressed in Section 4.
2. Training a neural network typically requires a lot of data, whereas macroeconomic datasets tend to be very small. For example, a monthly series of the consumer price index since the 1960s only has 800 observations. Section 5 suggests a solution.
3. It is important to be able to interpret the results of a model, in order to combat spurious correlations. Many powerful machine learning algorithms are often thought to be black boxes with very low interpretability. Section 6 tackles this challenge.

4 Analyzing factors with a Transformer

The first challenge is to find a way to use a Transformer for analyzing latent structures behind the data, instead of just predicting the observable variables.

Prediction extends the observable variables into the future. Long-term prediction of the macroeconomy is not feasible for several reasons. The data is limited, the underlying dynamic of the economy keeps changing over time, and people react to predictions. Most importantly, the future is drastically impacted by geopolitical and other surprises, which cannot be predicted.

Therefore it can be more interesting to *analyze* past and current unobserved variables and shocks, which impact the economic landscape. One example of such analysis is estimating a dynamic factor from data. A factor explains the common variation across many observable variables. When the factor is estimated from a suitable macroeconomic dataset, it can be interpreted as a coincident index of business cycle conditions, summarizing the current state of the economy.

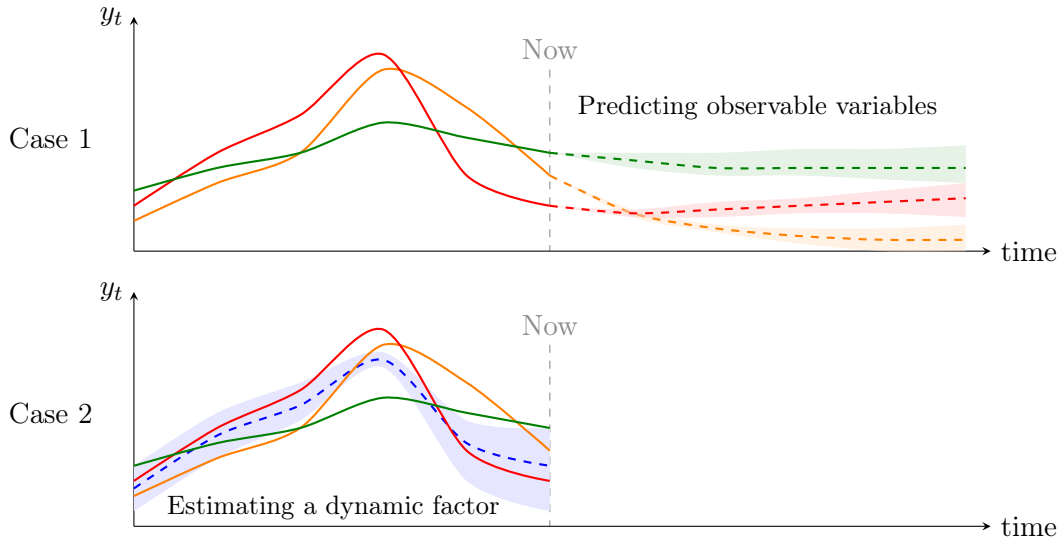


Figure 3: Case 1: Predicting observables, and Case 2: Estimating a dynamic factor. Transformers have mainly been used for prediction so far. Factor is a variable that affects the economy, but cannot be directly observed, such as “economic activity”.

The Transformer architecture explained in three resolutions

Level 1: The Transformer proposed here compresses the information in the data into one variable, and then expands it back out, Figure 4. The information that is distilled through the bottleneck is the factor estimate.

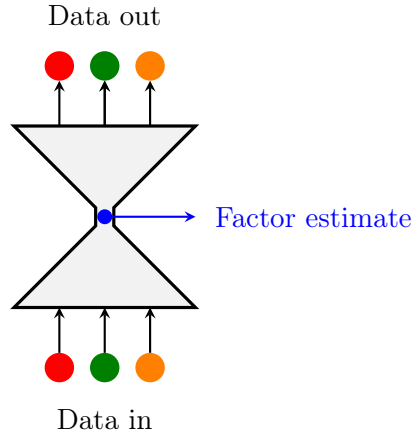


Figure 4: A very high level view of an autoencoder style Transformer.

Level 2: The macroeconomic data is first processed into a format that can be analyzed by a Transformer, see Figure 5. The data is fed to a State Encoder, which refines the factor estimate. The Transformer is then forced to heavily rely on the factor estimate when predicting the data onwards in the Measurement Encoder. During training the Transformer learns to construct the factor estimate from data in such a way that is useful for explaining the data. What is important here is that the factor is constructed without having to postulate any particular rigid model beforehand.

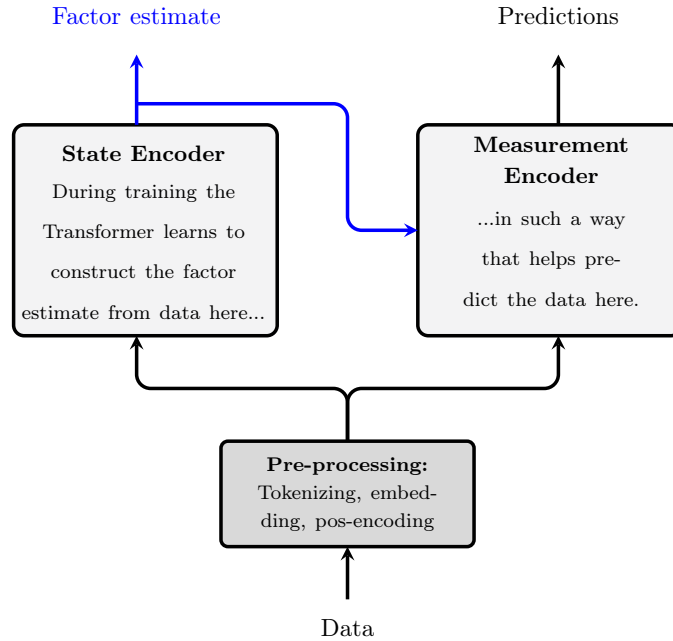


Figure 5: A medium resolution view of the Transformer for estimating factors.

Level 3a: Transformer Architecture. The Cross-Attention in the State Encoder gets Queries from the initial factor estimates, and Keys and Values from the data, in Figure 6. The output layer compresses the refined factor representations into one real number per lag position. The factor estimates are re-embedded to be used in the Measurement Encoder.

There, the Queries come from the data and the Keys and Values are projected from the factor estimates. The output layer again creates real number predictions, out of which only those corresponding to the $t + 1$ period are saved.

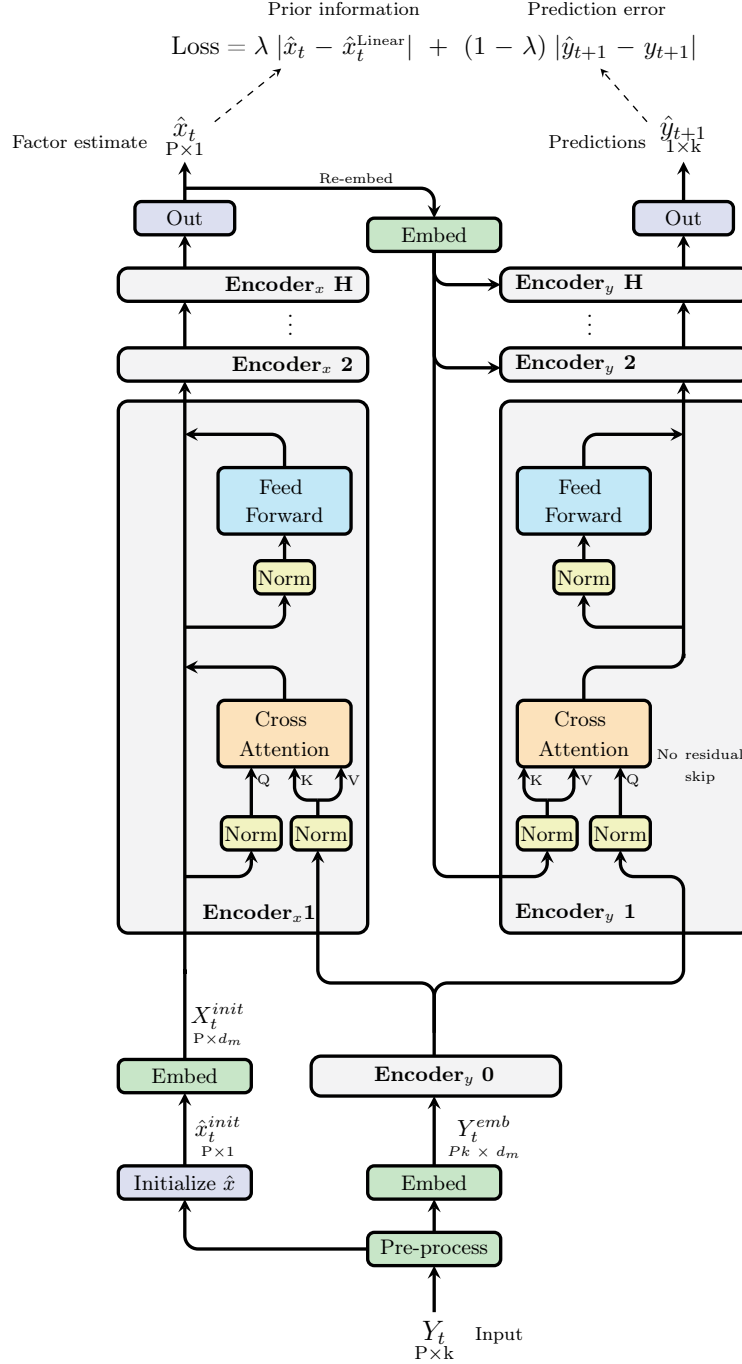


Figure 6: Detailed view of the Transformer architecture, for estimating dynamic factors. Both the factor estimate and predictions are supplied to the loss function. This is key to using prior information to guide the training in Section 5.

A bit more poetically: First the factor estimate asks questions and the data answers, and then the data asks questions and the factor estimate answers. The Attention modules are

the only places facilitating communication across tokens, whereas the feed-forward network and output projection don't pool. The width and depth of a Transformer have to be chosen before training. This fixes the number of parameters to be trained. Each particular set of parameter values corresponds to one possible model - some point in the set of all models in Figure 1. Initially the parameters are random, corresponding to a random point in the set. By training with data the Transformer updates the parameters in such a way that helps it better perform the objective given by the Loss function. This amounts to learning a model which is closer to the true model.

Level 3b: Data processing. Consider a time series dataset that consists of $i = 1, 2, \dots, k$ variables and $t = 1, 2, \dots, N$ time periods. The dataset is divided into three parts: a training set, which is used to update the parameter values; a validation set which is used to monitor how well the results generalize; and a test set, which is only looked at after the whole training process has concluded, to double check the generalization.

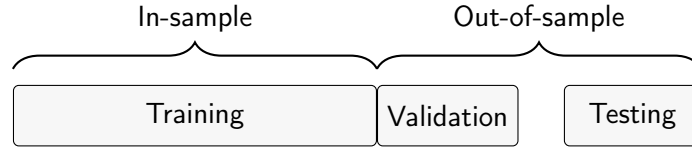


Figure 7: The data is divided into non-overlapping training, validation and test sets across the time dimension. Doing this to a macroeconomic time series dataset requires care, in order to get representative samples.

The Transformer processes data in blocks of P lags. A rolling window creates all possible information sets.

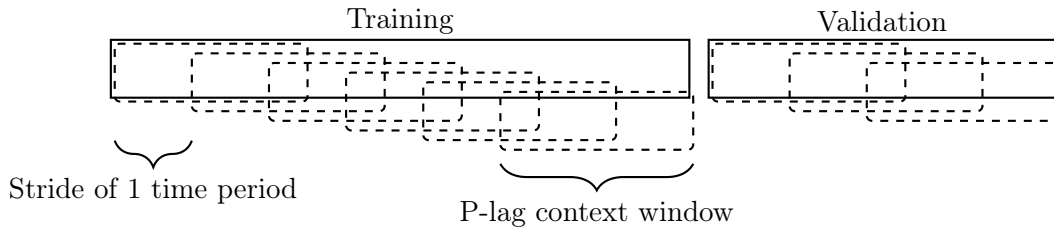


Figure 8: The training, validation and test sets are partitioned into P -lag context windows using a rolling window with a stride of 1.

Tokenizing partitions a P -lag context window Y_t of data into computational units, tokens. Defining what a token is has a large impact on the computational efficiency and interpretability of the results. A token is defined here as the value of one variable on one time period, $y_{ti} \in \mathbb{R}$. This choice maximizes interpretability of results with the Attention mechanism.

Embedding is a process of creating representations for tokens, which the Transformer can process. These representations are spacious information containers, into which the Transformer can encode more nuanced information, based on the dependencies and conditional patterns uncovered from the data.

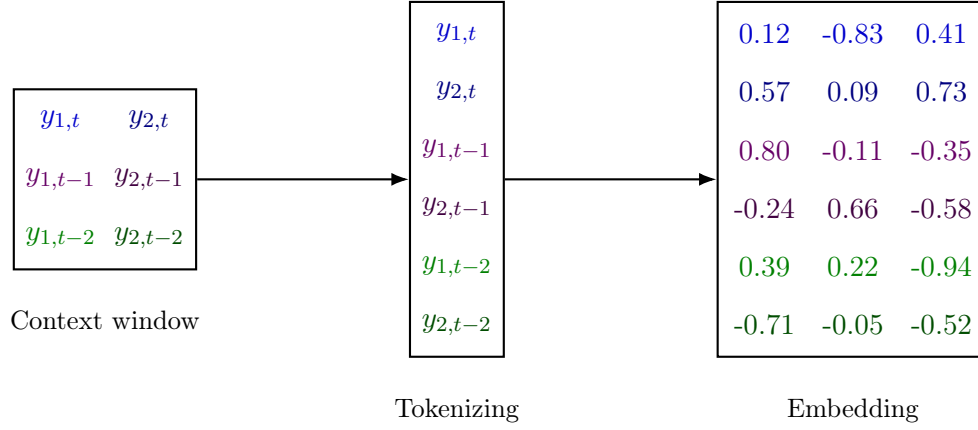


Figure 9: Pre-processing a context window with 2 variables and 2 lags.

Level 3c: Training procedure. From each subsequent context window, only the last period's factor estimate is saved. The Transformer can learn dynamic patterns within a context window, but each final factor estimate is based on the full information set. Context windows can be processed in parallel without recurrence, while there is still dynamic filtering within each window.

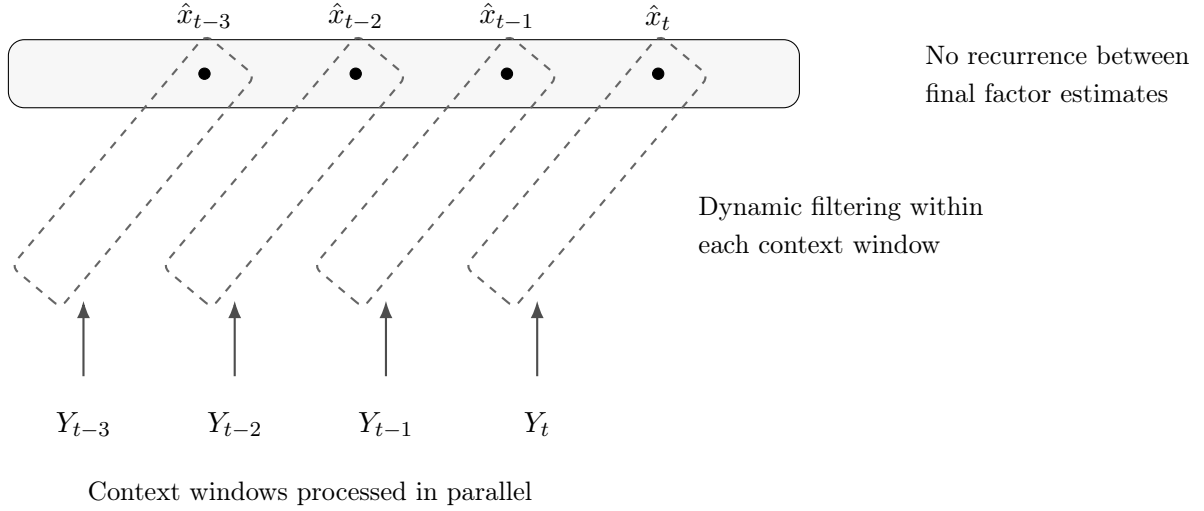


Figure 10: Training and inference is conducted in parallel, with local dynamic filtering within each context window.

The Transformer is trained multiple times for the same data. The final factor estimate is the piecewise mean of the training runs. The variation across runs measures the impact of parameter initialization. The real-time factor projections extend the estimates onward with a fixed information set.

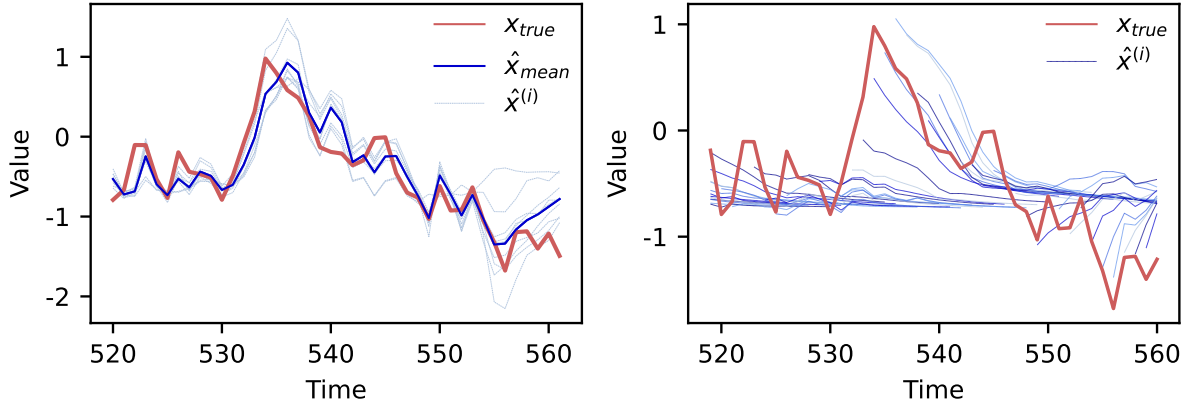


Figure 11: The left panel displays the estimated factor series in blue against a simulated ground truth in red. The panel on the right visualizes real time factor projections, which recursively feed the current predictions to the next period's inputs.

5 Prior information helps with small datasets

The second challenge shows that the strength of a Transformer can also be its Achilles' heel. A Transformer can be thought of as an automated abductive hypothesis/model tester. It can entertain complex ways to represent the data. When there's a lot of data, it is easy to see whether a model under consideration truly generalizes or not. With limited data it can be difficult for the Transformer to reject bad models. This means that the Transformer can converge to an unuseful model in the model space, left panel of Figure 12. More specifically, there is a lot of variation in quality across training runs. This is a problem in macroeconomics because of the scarcity of data, especially in the time dimension.

I suggest a way to use conventional models to guide the training. The Transformer is pulled towards them in the model space. Any conventional factor model can be used, but I demonstrate the functioning of prior information with a linear factor model. Often a linear model is a decent approximation of the true nonlinear model. The Transformer is given a useful neighborhood for where to start searching for a model, but it is not restricted to only look for linear models. This improves the results significantly on small datasets.

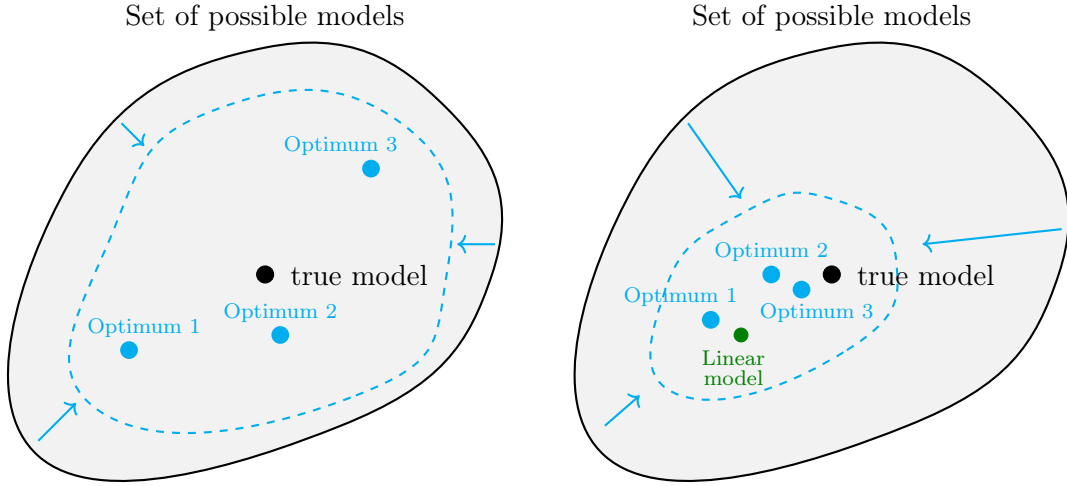


Figure 12: Left panel: With a small training dataset and without prior information the Transformer often finds suboptimal models. Right panel: A linear factor model pulls the Transformer towards a useful region in the model space.

Technically, the prior information is supplied with a new regularization term in the mean absolute error loss function. Both the factor estimates and the predictions enter the loss function, which guides the training of the Transformer, see Figure 6. The prior information regularizer minimizes the distance between the Transformer’s factor estimates $\hat{x}_T$ and the factor estimates from a chosen conventional model $\hat{x}_K$. The strength of the prior information is controlled with $\lambda \in [0, 1]$.

$$Loss = \underbrace{\lambda \frac{1}{P} \sum_{p=0}^{P-1} |\hat{x}_{T,t-p} - \hat{x}_{K,t-p}|}_{\text{Prior information}} + \underbrace{(1 - \lambda) \frac{1}{k} \sum_{i=1}^k |\hat{y}_{t+1,i} - y_{t+1,i}|}_{\text{Prediction error}}$$

The left panel of Figure 13 depicts a Transformer’s factor estimate $\hat{x}_T$ trained with modest prior information ($\lambda = 0.6$), reaching a 25% higher accuracy than the linear factor model $\hat{x}_K$. The panel on the right depicts a Transformer trained only on prior information ($\lambda = 1$), resulting in the same out-of-sample behavior as the linear factor model. the Transformer learns to emulate the chosen conventional model - without having to touch its nonlinear architecture!

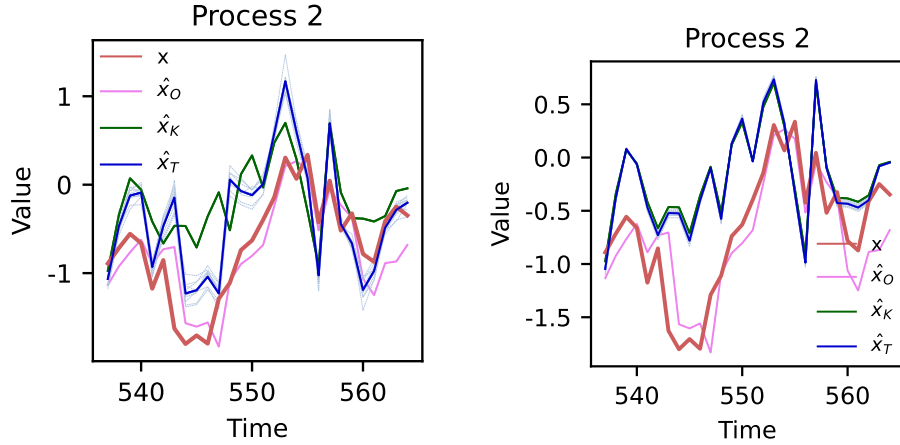


Figure 13: Impact of prior information on the results with simulated data. Left panel with weak prior information, right panel with only prior information. Simulated ground truth is shown in red by $\mathbf{x}$, and the factor estimate by the oracle particle filter in violet by $\hat{\mathbf{x}}_O$.

6 Interpreting results with Attention

The third challenge is related to the second one. There are plenty of bad models which can look good on limited data. It helps to check if the internal functioning of the model is in harmony with our understanding of the world. We want to be able to interpret WHY the model gave the output it did. This is easy with a model which is restricted to only search for constant linear relationships in the data. A simple linear model can be interpreted for example with “Unit increase in variable y_1 always indicates that the factor has increased by half of that.” However, this interpretability is acquired at the cost of forcing the data into a box it doesn’t naturally belong to. All of the information that doesn’t fit in has to be cut out. In some cases this is harmless or even useful, in others it can lead us astray.

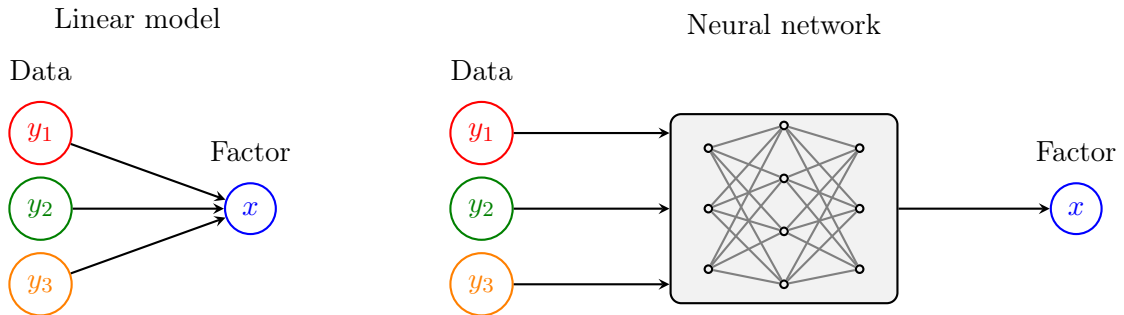


Figure 14: More complex models are trickier to interpret, because they allow the complex data to speak more freely.

Transformers have an advantage in interpretability over some other advanced machine learning techniques. A Transformer connects all inputs directly to each other, to find out what to pay attention to. The lack of recurrence can make Transformers easier to interpret

than many machine learning alternatives. We can visualize the Attention with a heatmap, where darker colors mean a higher level of attention, see Figure 15.

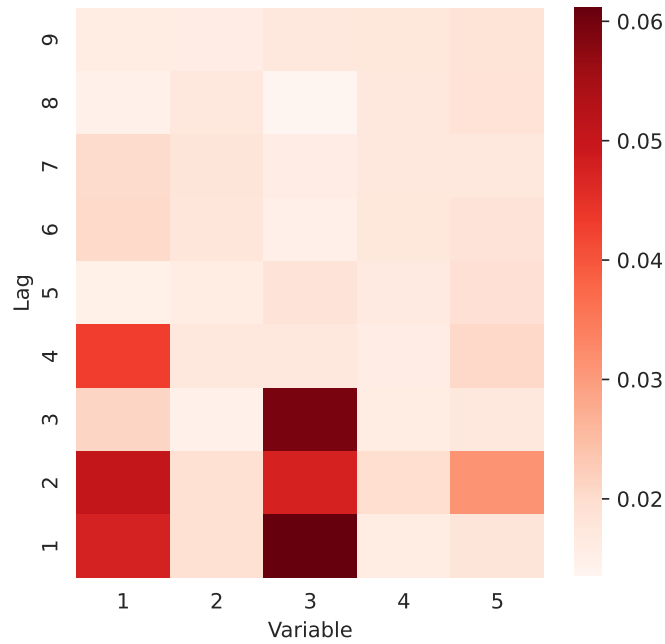


Figure 15: In this example there are five variables horizontally, each with nine lags on the vertical axis. The heatmap tells us that the Transformer deemed the recent values of variables one and three most important, for determining the factor estimate on a particular time period.

I customized time-series-specific tools to analyze how the attention patterns change over time. Figure 16 shows that the Transformer has initially found a steady pattern, which changes abruptly. Afterwards one of the variables rises in importance relative to others. This kind of change can indicate a regime switch. This tool could also help evaluate narrative checks, whether a shock was related to the supply or the demand side, based on which variables suddenly elevate in importance afterwards. An increase in the volatility of the Attention pattern could signal an increase in uncertainty.

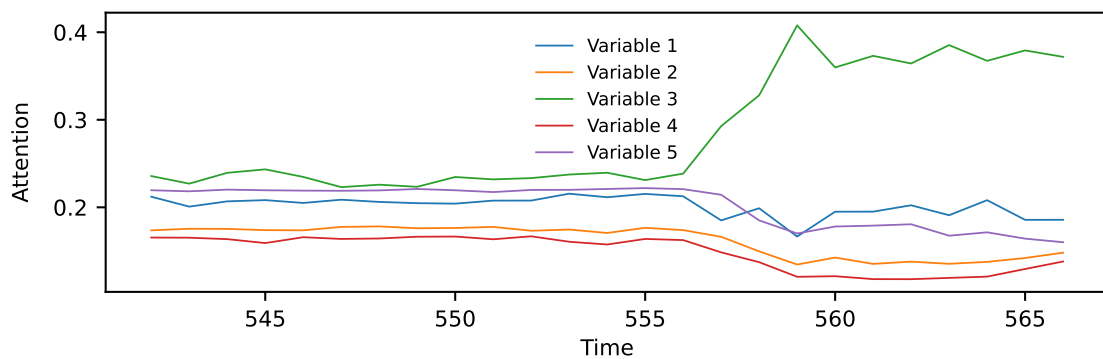


Figure 16: Time series plot showing the evolution of Attention patterns. A large change in the relative importance of variables hint at a regime switch.

We can also follow how the Attention pattern for input lags changes over time in Figure 17. Influential shocks can be inferred from a spike pattern: First the attention to most recent observations increases as the shock occurs. Then in the next period the attention is directed to the previous lag position, which now holds the information about the same shock. And so the shock propagates through the lag structure. If the Attention to the first lag position increases permanently, we could infer an increase in the uncertainty because in such a situation the most recent information increases in importance. When we zoom out to see how frequent these types of spike patterns are, we can assess how heavy-tailed the stochasticity of the underlying process is, Figure 18.

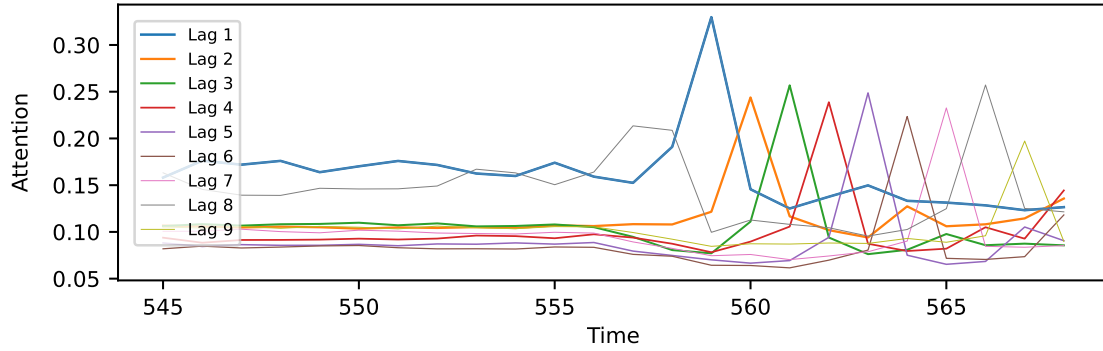


Figure 17: The spike pattern signals an influential shock that propagates through the lag structure.

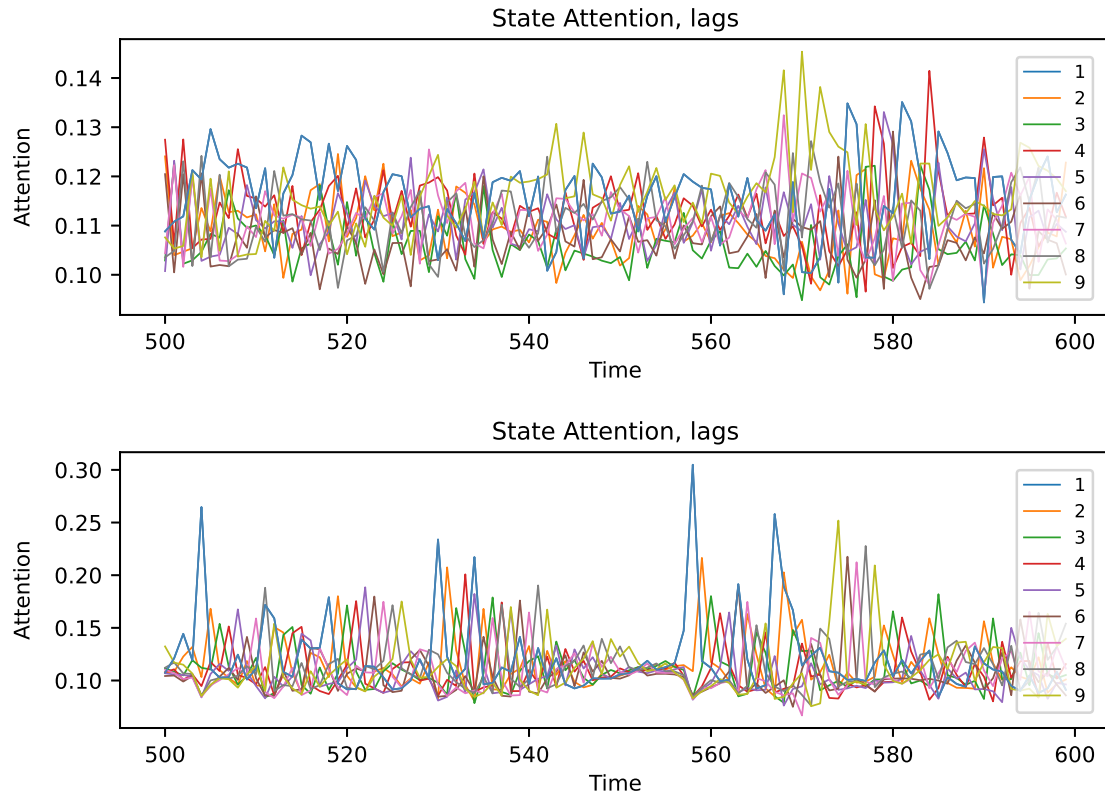


Figure 18: Frequent spike patterns in the lower panel hint at a heavy-tailed shock process. Upper panel describes a simulated Gaussian process, lower panel a t-distributed process with $df=10$.

References

- Snellman, O. (2025). Transformer Network for Macroeconometric Analysis. English. PhD thesis. University of Helsinki. ISBN: 978-952-84-1662-3.
- Vaswani, A. et al. (2017). Attention Is All You Need. *Advances in neural information processing systems* 30.